
UNDERSTANDING OF LONG TEXTS *

Girgorii Vinogradov
vinoggri@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic

ABSTRACT

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Keywords First keyword · Second keyword · More

1 Introduction

In recent years, the solutions to text classification, ranking, and retrieval problems in the field of NLP have changed significantly due to the development of language models, including LLM. We observe a key change in the method of text embedding, which is the transformation of text into a vector numerical representation. New language models based on transformer architecture, which use different variations of the attention mechanism, show excellent results in tasks where it is important to understand not only which words appear in the text and how important these words are, but also the overall context of the text. This approach allows for high-quality text embeddings, encoding more accurate, context-rich information into the final embedding vector.

Currently, the development of systems in which it is important to understand the meaning of text increasingly converges on the use of large language models (LLMs) that have a significant ability to “understand” the details of language and encode high-quality vector representations. This is achieved not only because these models have a large number of parameters, but also because they are pre-trained on a large amount of text, which allows them to understand most texts “out of the box.”

Unfortunately, such models are not a solution to all problems. More precisely, working with long texts using such models is difficult and represents a non-trivial task. In this work, we will examine this issue and try to draw some useful and reasonable conclusions about working with long texts in terms of understanding them through the creation of embedding vectors.

2 Embedding long texts with LLMs

As mentioned above, working with long texts and, more specifically, creating good, high-quality embeddings is a non-trivial task. The main problem is that LLMs have a limited context window, which means that they can only embed a limited sequence of tokens. For example, BERT [1] and XLM-RoBERTa [2] models, which are encoder-only models, have a context window of 512 tokens. This means that texts with a tokenized representation exceeding 512 tokens must be truncated to exactly this length. Note that the number of words in this approach will be even less than 512, since

**Citation:* Authors. Title. Pages.... DOI:000000/11111.

many words are encoded into more than one token, which can significantly degrade the quality of embedding unless all the key information is located in the first words of the long text sequence.

Another possible option for processing text so that it fits within the context length is to leave 512 tokens, in the case of BERT models, from the end of the text. There is also logic behind this, as many texts have a separate conclusion chapter that contains information about the entire text, only in a more condensed form.

The above methods are very simple for processing long texts. It is also worth noting that when trimming texts, we lose a large amount of information that can be very useful to us and that we want to encode in the final embedding vector. Another way to avoid rough text trimming is chunking. The idea is that we break the text into pieces that fit into the model's context. Next, we independently encode these parts using the model and obtain embeddings in the same way we would when trimming. After that, having obtained several embeddings, the number of which is equal to the number of chunks, we apply pooling (most often mean pooling or max pooling). In this way, we also preserve the information that is inside the text at any position. The problem with why this is also not an optimal text preprocessing strategy is that each embedding in the bank also has its own context and its own "importance" of that chunk of text relative to others. After applying, for example, mean pooling, we ignore the context and the importance of the chunks are mixed together. It is worth noting that this method requires more compute since instead of one pass, we are making multiple passes to process each chunk of text.

2.1 Strategies for long texts embedding

As discussed earlier, when we are dealing with long texts, we are trying to make embedding more context-rich. In this subsection, the strategies for long texts embedding will be discussed.

2.1.1 Text truncation

The first and most intuitive strategy to create an embedding for long text is to truncate a text, so it fits the context of the model [3, 4]. Usually, this is done by truncating the texts with a predefined number of tokens set to be the context length of the selected model.

2.1.2 Text chunking

The chunking strategy is a more accurate way of preprocessing long text input. This strategy involves splitting the long text input into chunks. Afterwards, those chunks are then passed to the LLM to obtain embeddings for each chunk the all those embeddings being aggregated into a final embedding.

One of the ways to aggregate chunks is to make a mean pooling. As previously discussed, this approach becomes suboptimal in scenarios where the information essential for accomplishing the task is confined to a limited segment of the input. In retrieval settings, for instance, the query-relevant data or the essential data to make an accurate classification may reside exclusively within one chunk. [4]

Another approach for aggregating the chunk embeddings is to design a layer that aggregates the text chunks. One of approaches to it is the hierarchical attention approach. This approach includes two stages of attention. Firstly, we split the texts into chunks (in [5] the chunking was performed based on sentences) and then passed them to the word (token) wise attention layers, which contextualize the first level of the text hierarchy. Then, in the second stage, the word-wise embeddings are passed to the transformer-like layer with the attention mechanism to contextualize the sentences. [5, 6]

In classification tasks, the selection of the most relevant or "best" chunk has also shown great results. [7]

3 Datasets

At this point of the work i have selected the dwzhu/LongEmbed, 2wikimqa dataset for the retrieval task, further i will also test the methods on classification tasks.

4 Methods

I have selected XLM-RoBERTa-large (Encoder) and Qwen3-Embedding (0.6B) (Decoder-based model) for testing the long texts strategies. In the notebook in my gitlab repository, the implementation of classes for encoders, based on each model as well as the code to evaluate on the 2wikimqa task.

5 Further Work

Further in work i will try to implement one of more sophisticated methods for chunk aggregation and try to generalize it to both classification and retrieval tasks.

6 Conclusion

Your conclusion here

Acknowledgments

This was was supported in part by.....

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [2] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. *CoRR*, abs/1911.02116, 2019.
- [3] Zican Dong, Tianyi Tang, Junyi Li, and Wayne Xin Zhao. A survey on long text modeling with transformers, 2025.
- [4] Hyunji Park, Yogarshi Vyas, and Kashif Shah. Efficient classification of long documents using transformers. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 702–709, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- [5] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In Kevin Knight, Ani Nenkova, and Owen Rambow, editors, *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489, San Diego, California, June 2016. Association for Computational Linguistics.
- [6] Ilias Chalkidis, Xiang Dai, Manos Fergadiotis, Prodromos Malakasiotis, and Desmond Elliott. An exploration of hierarchical attention transformers for efficient long document classification, 2022.
- [7] Mohit Tuteja and Daniel González Juclà. Long text classification using transformers with paragraph selection strategies. In Daniel Preotiuc-Pietro, Catalina Goanta, Ilias Chalkidis, Leslie Barrett, Gerasimos Spanakis, and Nikolaos Aletras, editors, *Proceedings of the Natural Legal Language Processing Workshop 2023*, pages 17–24, Singapore, December 2023. Association for Computational Linguistics.